

Семинар №11

Метапрограммирование

Мотивация

- Будем писать сложные “функции” не на уровне объектов, а на уровне типов
- Например, различные проверки на свойства типов: `is_class`, `is_copy_constructible`, ...

Explicit

- Начиная с C++ 20 explicit можно делать условным
- explicit (compile time bool expression)
- Похоже на noexcept
- Конструктор помечается explicit $\Leftrightarrow$ выражение true
- Помогает в [std::tuple](#): The constructor is explicit if and only if T_i is not copy-list-initializable from $\{ \}$ for at least one i .

Явное инстанцирование шаблонов

- Явно указываем компилятору инстанцировать шаблон с конкретными шаблонными параметрами
- Тогда мы сможем уменьшить время компиляции (так как инстанцируем только один раз)
- Уменьшаем объектные файлы
- Синтаксис:
 - `template void foo<int>(int);`
 - `template class Bar<int, double>;`
- Обычно пишут в .cpp файлах
- Возможно частичное инстанцирование (отдельных методов класса)
- Иногда реализацию шаблонных классов выносят в отдельный .ipp файл

“Кирпичики” для метапрограммирования

- `type_identity - type_identity<T>::type == T`
- `integral_constant – integral_constant<T, Val>::value == T tmp{Val}`
- `bool_constant – bool_constant<b>::value == b`
- `true_type – true_type::value == true`
- `false_type – false_type::value == false`
- `conjunction T1, ... Tn = T1::value && ... Tn::value`
- `Disjunction T1, ... Tn = T1::value || ... Tn::value`
- N.B.! Conjunction и disjunction должны работать “лениво” – инстанцировать только то, что необходимо

SFINAE

- SFINAE – substitution failure is not an error
- Если ошибка при подстановке типа в шаблонный параметр происходит при выборе перегрузки – то она не приводит к ошибке компиляции. Просто эта версия перегрузки не выбирается.
- Эта ошибка может быть в трех местах:
 - в типе возвращаемого значения
 - в типе аргумента функции
 - в шаблонных параметрах
- Но **НЕ** может быть в теле функции!